Data Structures for TAC

Quadruples

op	arg1	arg2	result

Some operators

$x = -y$ → | - | y | | x |

$x = y$ → | = | y | | x |

Function calls

call foo, 3 → | call | foo | 3 | |

x = call foo, 3 → | call | foo | 3 | x |

Labels

L1 : → | Label | | | L1 |

Function parameters

param arg1 → | param | arg1 | | |

Goto variations

goto L → | goto | | | L |

if x goto L → | if | x | | L |

ifFalse x goto L → | ifFalse | x | | L |

Returns

return → | return | | | |

return x → | return | x | | |

Eg: $a = -b * d + c + (-b) * d$

TAC

Triples

- Only three fields
- Results referred to by position rather than a temp var

(1) ⟶ pointer to instruction Is result

Disadvantage: If you move any instructions, all intrs reordered ⟶ all references updated

Eg: $a = -b * d + c + (-b) * d$

TAC

$t_1 = -b$
$t_2 = -b$
$t_3 = t_1 * d$
$t_4 = t_2 * d$
$t_5 = t_3 + c$
$t_6 = t_5 + t_4$
$a = t_6$

Triples

(1) | - | b | |
(2) | * | d | (1) |
(3) | + | c | (2) |
(4) | + | (3) | (2) |
(5) | = | a | (4) |

Indirect Triples

Contains a listing of pointers to triples instead the triples themselves

→ you can change the order of the list without having to update any numbering or references

Static Single-Assignment (SSA)

- Variant of TAC
- Every var declared only once; can be used multiple times ⟶ every value computed has unique assignment
- This simplifies several forms of code optimisation, including redundancy elimination

```
if (flag) x = -1; else x = 1;
y = x * a;
```

If different names used for x in if-else, which should be used in downstream instr?

ϕ-function

$x_3 = \phi(x_1, x_2)$

Combines multiple definitions of x that comes through more than one control path

$$\phi(x_1, x_2) = \begin{cases} x_1 & \text{if flag is True} \\ x_2 & \text{otherwise} \end{cases}$$

→ HOW?